Introduction I

Introduction II

Why generate projects at all

Most software templates are static: a directory tree is copied once, then diverges from its origin the moment a human edits it. That divergence is fine for a single project, but it defeats the purpose of a *template corpus* — a set of exemplars meant to be forked, specialized, and audited against a shared contract. The moment a fork edits away from its template, the template can no longer verify anything about the fork, and the fork can no longer prove it still satisfies the contract it was born from.

`template_autopoiesis` takes a different approach: instead of a directory that is copied once, it defines a **grammar** — a finite set of orthogonal *slots*, each with a finite set of *options* — and a pure function that maps a single integer seed plus that grammar to one specific, fully-formed child project. The grammar lives in `manuscript/config.yaml` under the `autopoiesis:` key and is parsed and validated by `parse_grammar()` in `src/grammar.py`. Two grammars that differ in even one option, one slot name, or one dependency string hash to different `grammar_hash` values, because `Grammar.grammar_hash` is the truncated SHA-256 of a